

Keep looking until you find it.

spring

- ★ 帮助读者快速熟悉Spring源码，以便对Spring源码进行扩展或修改，从而满足业务需求
- ★ 所有知识点均以HelloWorld级别示例为切入点，描述简单之后的复杂
- ★ 对于复杂逻辑的讲解采用剥洋葱似的方式，层层分解复杂度，便于读者理解和掌握

Spring

源码深度解析

● 郝佳 编著



人民邮电出版社
POSTS & TELECOM PRESS

Spring

源码深度解析

作为Java开源世界的第一开源框架，Spring已经成为事实上的Java EE开发标准。这些年，Spring不但得到了广泛的应用，还直接或间接地影响了众多JSR规范的制定。Spring的特性不仅覆盖了传统的Java EE开发，还拓展到移动开发、大数据、应用集成等领域，成为VMware云战略的重要组成部分。

Spring框架最根本的使命是简化Java开发，因此学习、研究、掌握Spring框架成为每一位Java开发人员的必修课。而阅读源码则是学习Spring的最好方式之一。优秀的开源源码都是集多年之经验，采众家之所长的，其中的意义妙不可言，现在我们试图把它们挖掘出来，并以一种易于理解的形式展现出来，让更多的读者从中受益。

本书重点

- Spring整体架构和环境搭建
- 容器的基本实现
- 默认标签的解析
- 自定义标签的解析
- Bean的加载
- 容器的功能扩展
- AOP
- 数据库连接JDBC
- 整合MyBatis
- 事务
- SpringMVC
- 远程服务
- Spring消息

阅读源码的魅力在于：

优秀源码的设计思想以及实现方式都是相通的，一旦掌握，就可以举一反三，一通百通；任何一款源码都经历了多年发展及提炼，其复杂程度和完美程度都远远超越我们日常开发的项目；很多源码都出自大师级开发人员之手，提供了各种先进的、精妙的解决问题的思路和方法，值得借鉴；在软件中，架构才是本质。阅读源码可以迅速看清项目整体框架的面貌，帮助读者快速提升开发技能。

更多关于Spring的信息可以访问  spring-source.com/community。



分类建议：计算机 / 程序设计 / Java
人民邮电出版社网址：www.ptpress.com.cn



ISBN 978-7-115-32568-6



ISBN 978-7-115-32568-6

定价：69.00 元

Spring

源码深度解析

● 郝佳 编著



人民邮电出版社
北京

图书在版编目 (CIP) 数据

Spring源码深度解析 / 郝佳编著. — 北京: 人民邮电出版社, 2013.9
ISBN 978-7-115-32568-6

I. ①S… II. ①郝… III. ①JAVA语言—程序设计
IV. ①TP312

中国版本图书馆CIP数据核字(2013)第165971号

内 容 提 要

本书从核心实现和企业应用两个方面,由浅入深、由易到难地对 Spring 源码展开了系统的讲解,包括 Spring 的设计理念和整体架构、容器的基本实现、默认标签的解析、自定义标签的解析、bean 的加载、容器的功能扩展、AOP、数据库连接 JDBC、整合 MyBatis、事务、SpringMVC、远程服务、Spring 消息服务等内容。

本书不仅介绍了使用 Spring 框架开发项目必须掌握的核心概念,还指导读者如何使用 Spring 框架编写企业级应用,并针对在编写代码的过程中如何优化代码、如何使得代码高效给出切实可行的建议,从而帮助读者全面提升实战能力。

本书语言简洁,示例丰富,可帮助读者迅速掌握使用 Spring 进行开发所需的各种技能。本书适合于已具有一定 Java 编程基础的读者,以及在 Java 平台下进行各类软件开发的开发人员、测试人员等。

-
- ◆ 编 著 郝 佳
责任编辑 杜 洁
责任印制 程彦红 杨林杰
 - ◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街 14 号
邮编 100061 电子邮件 315@ptpress.com.cn
网址 <http://www.ptpress.com.cn>
北京艺辉印刷有限公司印刷
 - ◆ 开本: 800×1000 1/16
印张: 24.75
字数: 545 千字
印数: 1—3 000 册
 - 2013 年 9 月第 1 版
2013 年 9 月北京第 1 次印刷
-

定价: 69.00 元

读者服务热线: (010) 67132692 印装质量热线: (010) 67129223
反盗版热线: (010) 67171154

源代码的重要性

Java 开发人员都知道，阅读源码是一个非常好的学习方式，在我们日常工作中或多或少都会接触一些开源代码，比如说最常用的 Struts、Hibernate、Spring，这些源码的普及与应用程度远远超过我们的想象，正因为很多人使用，也在推动着源码不断地去完善。这些优秀的源码中有着多年积淀下来的精华，这些精华是非常值得我们学习的，不管我们当前是什么水平，通过反复阅读源码能力能有所提升，小到对源码所提供的功能上的使用更加熟练，大到使我们的程序设计更加完美优秀。但是，纵观我们身边的人，能够做到通读源码的真的是少之又少，究其原因不外乎以下几点。

- ① 阅读源码绝对算得上是一件费时费力的工作，需要读者耗费大量的时间去完成。而作为开发人员，毕竟精力有限，实在没办法拿出太多的时间放在源码的阅读上。
- ② 源码的复杂性。任何一款源码经历了多年的发展与提炼，其复杂程度可想而知。当我们阅读源码的时候，大家都知道需要通过工具来跟踪代码的运行，进而去分析程序。但是，当代码过于复杂，环环相扣绕来绕去的时候，跟进了几十个甚至几百个函数后，这时我们已经不知道自己所处的位置了，不得不再重来，但是一一次又一次的，最终发现自己根本无法驾驭它，不得不放弃。
- ③ 有些源码发展多年，会遇到各种各样的问题，并对问题进行了解决，而这些问题有的对于我们来说甚至可以用莫名其妙来修饰，有时候根本想不出会在什么情况下会发生。我们选择各种查阅资料，查询无果，失去耐心，最终放弃。

无论基于什么样的原因，放弃阅读源码始终不是一个明智的选择，因为你失去了一个跟大师学习的机会。而且，当你读过几个源码之后你会发现，他们的思想以及实现方式是相通的。这就是开源的好处。随着各种开源软件的发展，各家都会融合别家优秀之处来不断完善自己，这样，到最后的結果就是所有的开源软件从设计上或者实现上都会

变得越来越相似，也就是说当你读完某个优秀源码后再去读另一个源代码，速度会有很大提升。

以我为例，Spring 是我阅读的第一个源码，几乎耗尽了将我近半年的时间，其中各种煎熬可想而知，但是当我读完 Spring 再去读 MyBatis 只用了两周时间。当然，暂且不论它们的复杂程度不同，至少我阅读的时候发现有很多相通的东西。当你第一次阅读的时候，你的重点一定是在源码的理解上，但是，当你读完第一个源码再去读下一个的时候，你自然而然地会带着批判或者说挑剔的眼光去阅读：为什么这个功能在我之前看的源码中是那样实现的，而在这里会是怎样实现的？这其中的道理在哪里，哪种实现方式更优秀呢？而通过这样的对比及探索，你会发现，自己的进步快得难以想象。

我们已经有些纠结了，既然阅读源码有那么多的好处，但是很多同学却因为时间或者能力的问题而不得不放弃，岂不是太可惜？为了解决这个问题，我撰写了本书，总结了自己的研究心得和实际项目经验，希望能对正在 Spring 道路上摸索的同仁们提供一些帮助。

本书特点

本书完全从开发者的角度去剖析源码，每一章都会提供具有代表性的实例，并以此为基础进行功能实现的分析，而不是采取开篇就讲解什么容器怎么实现、AOP 怎么实现之类的写法。在描述的过程中，本书尽可能地把问题分解，使用剥洋葱的方式一层一层地将逻辑描述清楚，帮助读者由浅入深地进行学习，并把这些难点和问题各个击破，而不是企图一下让读者理解一个复杂的逻辑。

在阅读源码的过程中，我们难免会遇到各种各样的生僻功能，这些功能在特定的场合会非常有用，但是可能多数情况下并不是很常用，甚至都查阅不到相关的使用资料。本书中重点针对这种情况提供了相应的实用示例，让读者更加全面地了解 Spring 所提供的功能，对代码能知其然还知其所以然。

本书按照每章所提供的示例跟踪 Spring 源码的流程，尽可能保证代码的连续性，使读者的思维不被打乱，让读者看到 Spring 的执行流程，旨在尽量使读者在阅读完本书后即使在不阅读 Spring 源码的情况下也可以对 Spring 源码进行优化，甚至通过扩展源码来满足业务需求，这对开发人员来说是一个很高的要求。本书就希望能帮助读者全面提升实战能力。

本书结构

本书分为两部分：核心实现和企业应用。

- 第一部分 核心实现（第 1~7 章）：是 Spring 功能的基础，也是企业应用部分的基础，主要对容器以及 AOP 功能实现做了具体的分析，如果读者之前没有接触过 Spring 源代码，建议认真阅读这个部分，否则阅读企业应用部分时

会比较吃力。

- 第二部分 企业应用 (第 8~13 章): 在核心实现部分的基础上围绕企业应用常用的模块进行讨论, 这些模块包括 Spring 整合 JDBC、Spring 整合 MyBatis、事务、SpringMVC、远程服务、Spring 消息服务等, 旨在帮助读者在日常开发中更加高效地使用 Spring。

本书适用的 Spring 版本

截至完稿, Spring 已经发布了 4.0.0.M1 版本。本书虽然是基于 Spring 3.2 版本编写的, 但所讨论的内容都属于 Spring 的基础和常用的功能, 这些功能都经过长时间、大量用户的验证, 已经非常成熟, 改动的可能性相对较小。而且从目前 Spring 的功能规划来看, 本书所涉及的内容并不在 Spring 未来改动的范围内, 因此在未来的很长一段时间内本书都不会过时的。

感谢

创作的过程是痛苦的, 持续时间也远远超乎了我的想象, 而本以为自己对 Spring 已经非常的熟悉, 但是在写作的过程中还是会遇到各种各样的问题, 但是我很幸运我能坚持下来, 在这里我首先应该感谢爸爸妈妈, 虽然他们不知道儿子在忙忙碌碌地写些什么, 但是他们对我始终如一的支持与鼓励使我更加坚定信心, 在这里祝他们身体健康, 同时还要感谢我最好的朋友孙亚超在我低落时给予我的关心与问候, 当然要感谢的还有张雨绮同学, 虽然并不是明星, 但是却有着堪比明星般的美丽笑容, 与她在一起的讨论总是让我受益匪浅, 同时也感谢妹子王晶对稿件提供的建议与意见。最后感谢郭维云、郝云勃、郝俊、李兴全、梁晓颖、陈森、孙伟超、王璐、刘瑞、单明、姚佳林、闫微微、李娇、时宇、李平、唐广亮、刘阳、黄思文、金施源等在整个编写过程中给予的支持与帮助。

联系作者

在编写本书过程中, 以“够用就好”为原则, 尽量覆盖到 Spring 开发的常用功能。所有观点都出自作者的个人见解, 疏漏、错误之处在所难免, 欢迎大家指正。读者如果有好的建议或者学习本书过程中遇到问题, 请发送邮件到 haojia_007@163.com, 希望能够与大家一起交流和进步。

在看得见的地方学习知识, 在看不到的地方学习智慧。祝愿大家在 Spring 的学习道路上顺风顺水。

作者

2013 年 7 月

作者简介



郝佳

计算机专业硕士学位，曾发表过多篇论文先后被 EI、SCI 收录，2008 年辽宁省教育厅科技计划项目研究人之一；长期奋斗于 J2EE 领域，曾任职于某互联网公司软件架构师，擅长系统的性能优化，目前正在投身于开发一款基于 Java 并发多线程管理的开源框架；热衷于研究各种优秀的开源代码并从中进行总结，从而实现个人技能的提高，尤其对 Spring、Hibernate、MyBatis、JMS、Tomcat 等源码有着深刻的理解和认识。

目录

第一部分 核心实现

第 1 章 Spring 整体架构和环境

搭建 2

- 1.1 Spring 的整体架构 2
- 1.2 环境搭建 4
 - 1.2.1 安装 GitHub 4
 - 1.2.2 安装 Gradle 5
 - 1.2.3 下载 Spring 6

第 2 章 容器的基本实现 10

- 2.1 容器基本用法 10
- 2.2 功能分析 11
- 2.3 工程搭建 12
- 2.4 Spring 的结构组成 13
 - 2.4.1 beans 包的层级结构 13
 - 2.4.2 核心类介绍 13
- 2.5 容器的基础
 - XmlBeanFactory 17
 - 2.5.1 配置文件封装 18
 - 2.5.2 加载 Bean 21
- 2.6 获取 XML 的验证模式 24
 - 2.6.1 DTD 与 XSD 区别 24
 - 2.6.2 验证模式的读取 26
- 2.7 获取 Document 28

- 2.7.1 EntityResolver 用法 29

2.8 解析及注册

BeanDefinitions 31

- 2.8.1 profile 属性的使用 32
- 2.8.2 解析并注册
 - BeanDefinition 33

第 3 章 默认标签的解析 35

3.1 bean 标签的解析及注册 35

- 3.1.1 解析 BeanDefinition 37
- 3.1.2 AbstractBeanDefinition 属性 55
- 3.1.3 解析默认标签中的自定义标签元素 58
- 3.1.4 注册解析的 BeanDefinition 60
- 3.1.5 通知监听器解析及注册完成 63

3.2 alias 标签的解析 63

3.3 import 标签的解析 65

3.4 嵌入式 beans 标签的解析 67

第 4 章 自定义标签的解析 68

4.1 自定义标签使用 69

- 4.2 自定义标签解析 71
 - 4.2.1 获取标签的命名空间 72
 - 4.2.2 提取自定义标签处理器 72
 - 4.2.3 标签解析 74
- 第5章 bean的加载 78
 - 5.1 FactoryBean的使用 83
 - 5.2 缓存中获取单例 bean 85
 - 5.3 从 bean 的实例中获取对象 86
 - 5.4 获取单例 90
 - 5.5 准备创建 bean 92
 - 5.5.1 处理 override 属性 93
 - 5.5.2 实例化的前置处理 94
 - 5.6 循环依赖 96
 - 5.6.1 什么是循环依赖 96
 - 5.6.2 Spring 如何解决循环依赖 96
 - 5.7 创建 bean 100
 - 5.7.1 创建 bean 的实例 103
 - 5.7.2 记录创建 bean 的 ObjectFactory 112
 - 5.7.3 属性注入 115
 - 5.7.4 初始化 bean 124
 - 5.7.5 注册 DisposableBean 128
- 第6章 容器的功能扩展 129
 - 6.1 设置配置路径 130
 - 6.2 扩展功能 130
 - 6.3 环境准备 132
 - 6.4 加载 BeanFactory 133
 - 6.4.1 定制 BeanFactory 135
 - 6.4.2 加载 BeanDefinition 136
 - 6.5 功能扩展 137
 - 6.5.1 增加 SPEL 语言的支持 138
 - 6.5.2 增加属性注册编辑器 139
 - 6.5.3 添加 ApplicationContextAwareProcessor 处理器 144
 - 6.5.4 设置忽略依赖 146
 - 6.5.5 注册依赖 146
 - 6.6 BeanFactory 的后处理 146
 - 6.6.1 激活注册的 BeanFactoryPostProcessor 147
 - 6.6.2 注册 BeanPostProcessor 153
 - 6.6.3 初始化消息资源 156
 - 6.6.4 初始化 ApplicationEventMulticaster 159
 - 6.6.5 注册监听器 161
 - 6.7 初始化非延迟加载单例 162
 - 6.8 finishRefresh 165
- 第7章 AOP 167
 - 7.1 动态 AOP 使用示例 167
 - 7.2 动态 AOP 自定义标签 169
 - 7.2.1 注册 AnnotationAwareAspectJAutoProxyCreator 170
 - 7.3 创建 AOP 代理 173
 - 7.3.1 获取增强器 176
 - 7.3.2 寻找匹配的增强器 186
 - 7.3.3 创建代理 187
 - 7.4 静态 AOP 使用示例 201
 - 7.5 创建 AOP 静态代理 203
 - 7.5.1 Instrumentation 使用 203
 - 7.5.2 自定义标签 207
 - 7.5.3 织入 209
- 第二部分 企业应用
- 第8章 数据库连接 JDBC 214
 - 8.1 Spring 连接数据库程序实现 (JDBC) 215
 - 8.2 save/update 功能的实现 217
 - 8.2.1 基础方法 execute 219
 - 8.2.2 Update 中的回调函数 223
 - 8.3 query 功能的实现 225

8.4 queryForObject 229

第9章 整合 MyBatis 231

9.1 MyBatis 独立使用 231

9.2 Spring 整合 MyBatis 235

9.3 源码分析 237

9.3.1 sqlSessionFacory 创建 237

9.3.2 MapperFactoryBean 的
创建 241

9.3.3 MapperScannerConfigurer 244

第10章 事务 254

10.1 JDBC 方式下的事务使用
示例 254

10.2 事务自定义标签 257

10.2.1 注册 InfrastructureAdvisor
AutoProxyCreator 25710.2.2 获取对应 class/method 的
增强器 261

10.3 事务增强器 269

10.3.1 创建事务 271

10.3.2 回滚处理 281

10.3.3 事务提交 287

第11章 SpringMVC 291

11.1 SpringMVC 快速体验 291

11.2 ContextLoaderListener 295

11.2.1 ServletContextListener 的
使用 29511.2.2 Spring 中的 ContextLoader
Listener 296

11.3 DispatcherServlet 300

11.3.1 servlet 的使用 301

11.3.2 DispatcherServlet 的
初始化 30211.3.3 WebApplicationContext 的
初始化 30411.4 DispatcherServlet 的逻辑
处理 32011.4.1 MultipartContent 类型的
request 处理 32611.4.2 根据 request 信息寻找
对应的 Handler 32711.4.3 没找到对应的 Handler 的
错误处理 33111.4.4 根据当前 Handler 寻找对
应的 HandlerAdapter 331

11.4.5 缓存处理 332

11.4.6 HandlerInterceptor 的
处理 333

11.4.7 逻辑处理 334

11.4.8 异常视图的处理 334

11.4.9 根据视图跳转页面 335

第12章 远程服务 340

12.1 RMI 340

12.1.1 使用示例 340

12.1.2 服务端实现 342

12.1.3 客户端实现 350

12.2 HttpInvoker 355

12.2.1 使用示例 356

12.2.2 服务端实现 357

12.2.3 客户端实现 361

第13章 Spring 消息 367

13.1 JMS 的独立使用 367

13.2 Spring 整合 ActiveMQ 369

13.3 源码分析 371

13.3.1 JmsTemplate 372

13.3.2 监听器容器 376

第一部分 核心实现



第 1 章 Spring 整体架构和环境搭建

Spring 是于 2003 年兴起的一个轻量级的 Java 开源框架，由 Rod Johnson 在其著作《Expert One-On-One J2EE Development and Design》中阐述的部分理念和原型衍生而来。Spring 是为了解决企业应用开发的复杂性而创建的，它使用基本的 JavaBean 来完成以前只可能由 EJB 完成的事情。然而，Spring 的用途不仅限于服务器端的开发。从简单性、可测试性和松耦合的角度而言，任何 Java 应用都可以从 Spring 中受益。

1.1 Spring 的整体架构

Spring 框架是一个分层架构，它包含一系列的功能要素，并被分为大约 20 个模块，如图 1-1 所示。

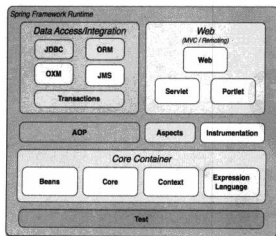


图 1-1 Spring 整体架构图

这些模块被总结为以下几部分。

(1) Core Container.

Core Container (核心容器) 包含有 Core、Beans、Context 和 Expression Language 模块。

Core 和 Beans 模块是框架的基础部分, 提供 IoC (转控制) 和依赖注入特性。这里的基础概念是 BeanFactory, 它提供对 Factory 模式的经典实现来消除对程序性单例模式的需要, 并真正地允许你从程序逻辑中分离出依赖关系和配置。

- Core 模块主要包含 Spring 框架基本的核心工具类, Spring 的其他组件都要使用到这个包里的类, Core 模块是其他组件的基本核心。当然你也可以在自己的应用系统中使用这些工具类。
- Beans 模块是所有应用都要用到的, 它包含访问配置文件、创建和管理 bean 以及进行 Inversion of Control / Dependency Injection (IoC/DI) 操作相关的所有类。
- Context 模块构建于 Core 和 Beans 模块基础之上, 提供了一种类似于 JNDI 注册器的框架式的对象访问方法。Context 模块继承了 Beans 的特性, 为 Spring 核心提供了大量扩展, 添加了对国际化 (例如资源绑定)、事件传播、资源加载和对 Context 的透明创建的支持。Context 模块同时也支持 J2EE 的一些特性, 例如 EJB、JMX 和基础的远程处理。ApplicationContext 接口是 Context 模块的关键。
- Expression Language 模块提供了一个强大的表达式语言用于在运行时查询和操纵对象。它是 JSP 2.1 规范中定义的 unified expression language 的一个扩展。该语言支持设置/获取属性的值, 属性的分配, 方法的调用, 访问数组上下文 (accessing the context of arrays)、容器和索引器、逻辑和算术运算符、命名变量以及从 Spring 的 IoC 容器中根据名称检索对象。它也支持 list 投影、选择和一般的 list 聚合。

(2) Data Access/Integration.

Data Access/Integration 层包含有 JDBC、ORM、OXM、JMS 和 Transaction 模块, 其中:

- JDBC 模块提供了一个 JDBC 抽象层, 它可以消除冗长的 JDBC 编码和解析数据库厂商特有的错误代码。这个模块包含了 Spring 对 JDBC 数据访问进行封装的所有类。
- ORM 模块为流行的对象-关系映射 API, 如 JPA、JDO、Hibernate、iBatis 等, 提供了一个交互层。利用 ORM 封装包, 可以混合使用所有 Spring 提供的特性进行 O/R 映射。如前边提到的简单声明性事物管理。

Spring 框架插入了若干个 ORM 框架, 从而提供了 ORM 的对象关系工具, 其中包括 JDO、Hibernate 和 iBatisSQL Map。所有这些遵从 Spring 的通用事务和 DAO 异常层次结构。

- OXM 模块提供了一个对 Object/XML 映射实现的抽象层, Object/XML 映射实现包括 JAXB、Castor、XMLBeans、JiBX 和 XStream。
- JMS (Java Messaging Service) 模块主要包含了一些制造和消费消息的特性。
- Transaction 模块支持编程和声明性的事物管理, 这些事物类必须实现特定的接口, 并且对所有的 POJO 都适用。

(3) Web。

Web 上下文模块建立在应用程序上下文模块之上,为基于 Web 的应用程序提供了上下文。所以, Spring 框架支持与 Jakarta Struts 的集成。Web 模块还简化了处理多部分请求以及将请求参数绑定到域对象的工作。Web 层包含了 Web、Web-Servlet、Web-Struts 和 Web-Portlet 模块,具体说明如下。

- Web 模块:提供了基础的面向 Web 的集成特性。例如,多文件上传、使用 servlet listeners 初始化 IoC 容器以及一个面向 Web 的应用上下文。它还包含 Spring 远程支持中 Web 的相关部分。
- Web-Servlet 模块 web.servlet.jar:该模块包含 Spring 的 model-view-controller (MVC) 实现。Spring 的 MVC 框架使得模型范围内的代码和 web forms 之间能够清楚地分离开来,并与 Spring 框架的其他特性集成在一起。
- Web-Struts 模块:该模块提供了对 Struts 的支持,使得类在 Spring 应用中能够与一个典型的 Struts Web 层集成在一起。注意,该支持在 Spring 3.0 中是 deprecated 的。
- Web-Portlet 模块:提供了用于 Portlet 环境和 Web-Servlet 模块的 MVC 的实现。

(4) AOP。

AOP 模块提供了一个符合 AOP 联盟标准的面向切面编程的实现,它让你可以定义例如方法拦截器和切点,从而将逻辑代码分开,降低它们之间的耦合性。利用 source-level 的元数据功能,还可以将各种行为信息合并到你的代码中,这有点像 .Net 技术中的 attribute 概念。

通过配置管理特性, Spring AOP 模块直接将面向切面的编程功能集成到了 Spring 框架中,所以可以很容易地使 Spring 框架管理的任何对象支持 AOP。Spring AOP 模块为基于 Spring 的应用程序中的对象提供了事务管理服务。通过使用 Spring AOP,不用依赖 EJB 组件,就可以将声明性事务管理集成到应用程序中。

- Aspects 模块提供了对 AspectJ 的集成支持。
- Instrumentation 模块提供了 class instrumentation 支持和 classloader 实现,使得可以在特定的应用服务器上使用。

(5) Test。

Test 模块支持使用 JUnit 和 TestNG 对 Spring 组件进行测试。

12 环境搭建

Spring 已经将源码从 svn 迁移到了 GitHub。而且也改为基于 Gradle 的构建来构建项目,它取代了之前的 Ant+Ivy 系统,所以要构建 Spring 源码环境首先要安装 GitHub 以及 Gradle。

1.2.1 安装 GitHub

首先读者需要到 GitHub 官网去下载安装包,其中 Windows 系统对应的版本下载地址为:

<http://windows.github.com/>，下载后双击进行安装。安装成功后，快捷菜单中会出现 GitHub 的菜单，如图 1-2 所示。

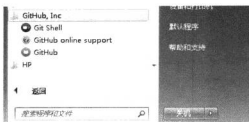


图 1-2 HitHub 安装成功后的启动菜单

1.2.2 安装 Gradle

Gradle 是一个基于 Groovy 的构建工具，它使用 Groovy 来编写构建脚本，支持依赖管理和多项目创建，类似 Maven，但比其更加简单轻便。Gradle 为 Ivy 提供了一个 layer，提供了 build-by-convention 集成，而且它还让你获得许多类似 Maven 的功能。你可以从 <http://www.gradle.org/downloads> 页面下载 Gradle，下载后将文件解压放到指定目录中（笔者放在了 C:\Program Files 目录下），然后开始进行环境变量的配置。

(1) 根据对应目录创建 GRADLE_HOME 系统变量，如图 1-3 所示。

(2) 将系统变量加入到 path 中，如图 1-4 所示。



图 1-3 创建对应于 Gradle 的系统变量

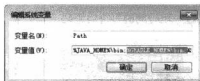


图 1-4 将 Gradle 对应的系统变量加入 path 中

(3) 测试。

当完成系统变量的配置后打开命令窗口输入命令“gradle -version”，如果安装成功会出现 Gradle 对应的版本信息，如图 1-5 所示。



图 1-5 测试 Gradle 的环境变量配置

1.2.3 下载 Spring

因为 Spring 源码是通过 GitHub 进行管理的, 所以我们首先打开 GitHub, 单击快捷菜单中的“Git Shell”选项, 如图 1-6 所示。

打开 GitHub 后, 你可以通过 `cd` 命令将当前操作目录转换到我们想要存储源码的目录, 例如, 想要将下载的源码存储到 `e:\test` 下, 则可以执行“`cd e:\test`”。

输入以下命令:

```
git clone git://github.com/SpringSource/Spring-framework.git
```

其中“`git://github.com/SpringSource/Spring-framework.git`”为 Spring 的源码地址。执行命令后便进入源码下载状态, 如图 1-7 所示。

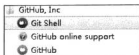


图 1-6 启动 GitHub 启动菜单



图 1-7 使用 GitHub 开始下载源码

经过一段时间的等待后源码下载结束, 窗口状态如图 1-8 所示。

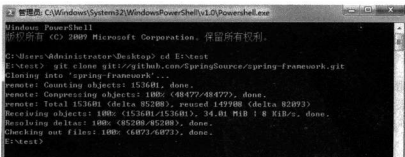


图 1-8 源码下载结束的窗口显示

而这时候我们去查看, 对应的文件夹下已经存在了相应的源码信息, 如图 1-9 所示。



图 1-9 下载的 Spring 源码

但是当前的源码并不可以直接导入 Eclipse 中，我们还需要将源码转换为 Eclipse 可以读取的形式。网上有各种各样的方法，其中出现最多的是告诉大家将所有工程一次性的编译、导入，但是笔者并不推荐这样的方式，因为这样会耗费大量的时间，而且当存在编译错误的时候你不得不重新编译。笔者建议只对我们感兴趣的工程进行 Eclipse 工程转换，比如我们想要查看 Spring 事务部分的源码，打开命令窗口，将当前目录切换至源码所在目录，例如，这里是 Spring-tx 文件夹下，执行命令“gradle cleanidea eclipse”，当窗口出现如下状态说明已经开始执行转换过程，如图 1-10 所示。

```
D:\test\spring-framework\spring-tx>gradle cleanidea eclipse
:buildSrc:compileJava UP-TO-DATE
:buildSrc:compileGroovy UP-TO-DATE
:buildSrc:processResources UP-TO-DATE
```

图 1-10 Spring 源码转换至 eclipse 工程

经过一段时间后转换成功，如图 1-11 所示。

这时，我们再查看对应的文件夹会发现，已经出现了作为 Eclipse 工程所必须的.project 与.classpath 文件了，如图 1-12 所示。

打开 Eclipse，将工程导入，导入后如图 1-13 所示。

```

:spring-tx:eclipseJdtPrepare
:spring-tx:eclipseJdt
:spring-tx:eclipseProject
:spring-tx:eclipseSettings
:spring-tx:eclipseVstComponent
:spring-tx:eclipse
BUILD SUCCESSFUL
Total time: 52.511 secs
D:\test\spring-framework\spring-tx>

```

图 1-11 Spring 源码成功转换至 eclipse 工程

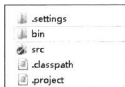


图 1-12 转换至 Eclipse 工程后的 Spring 源码结构



图 1-13 导入 Eclipse 后的源码工程

你会发现工程名称前面有一个感叹号，这说明存在错误。查看依赖包及工程，会看到当前工程所依赖的包已经完全导入，没有问题，工程所依赖的 JAR 如图 1-14 所示。

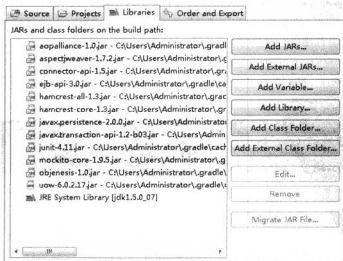


图 1-14 工程依赖的 JAR

但是，查看依赖的 Projects 时发现，当前工程还要依赖于其他 Spring 中的 6 个工程，这时，读者可以选择以同样的方式继续导入源码工程，或者，直接找到对应的 JAR 加入编译路径，工程所依赖的 Projects 如图 1-15 所示。

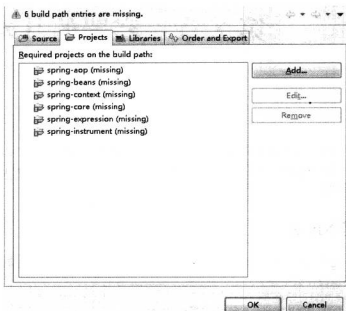


图 1-15 工程所依赖的 Projects

第 2 章 容器的基本实现

源码分析是一件非常煎熬非常有挑战性的任务，你准备好开始战斗了吗？

在正式开始分析 Spring 源码之前，我们有必要先来回顾一下 Spring 中最简单的用法，尽管我相信您已经对这个例子非常熟悉了。

2.1 容器基本用法

bean 是 Spring 中最核心的东西，因为 Spring 就像是个大水桶，而 bean 就像是容器中的水，水桶脱离了水便也没什么用处了，那么我们先看看 bean 的定义。

```
public class MyTestBean {
    private String testStr = "testStr";

    public String getTestStr() {
        return testStr;
    }

    public void setTestStr(String testStr) {
        this.testStr = testStr;
    }
}
```

很普通，bean 没有任何特别之处，的确，Spring 的目的就是让我们的 bean 能成为一个纯粹的 POJO，这也是 Spring 所追求的。接下来看看配置文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/Spring-beans.xsd">

    <bean id="myTestBean" class="bean.MyTestBean"/>

</beans>
```

在上面的配置中我们看到了 bean 的声明方式, 尽管 Spring 中 bean 的元素定义着 N 种属性来支撑我们业务的各种应用, 但是我们只要声明成这样, 基本上就已经可以满足我们的大多数应用了。好了, 你可能觉得还有什么, 但是, 真没了, Spring 的入门示例到这里已经结束, 我们可以写测试代码测试了。

```
@SuppressWarnings("deprecation")
public class BeanFactoryTest {

    @Test
    public void testSimpleLoad(){
        BeanFactory bf = new XmlBeanFactory(new ClassPathResource("beanFactoryTest.xml"));
        MyTestBean bean = (MyTestBean) bf.getBean("myTestBean");
        assertEquals("testStr", bean.getTestStr());
    }
}
```

相信聪明的读者会很快看到我们期望的结果: 在 Eclipse 中显示了 Green Bar。

直接使用 BeanFactory 作为容器对于 Spring 的使用来说并不多见, 甚至是甚少使用, 因为在企业级的应用中大多数都会使用的是 ApplicationContext (后续章节我们会介绍它们之间的区别), 这里只是用于测试, 让读者更快更好地分析 Spring 的内部原理。

OK, 我们又复习了一遍 Spring, 你是不是会很不屑呢? 这样的小例子没任何挑战性。嗯, 确实, 这样的使用是过于简单了, 但是本书的目的并不是介绍如何使用 Spring, 而是帮助您更好地了解 Spring 的内部原理。读者可以自己先想想, 上面的一句简单代码都执行了什么样的逻辑呢? 这样一句简单代码其实在 Spring 中执行了太多太多的逻辑, 即使笔者用半本书的文字也只能介绍它的大致原理。那么就让我们快速的进入分析状态吧。

2.2 功能分析

现在我们可以来好好分析一下上面测试代码的功能, 来探索上面的测试代码中 Spring 究竟帮助我们完成了什么工作? 不管之前你是否使用过 Spring, 当然, 你应该使用过的, 毕竟本书面用的是对 Spring 有一定使用经验的读者, 你都应该能猜出来, 这段测试代码完成的功能无非就是以下几点。

- (1) 读取配置文件 beanFactoryTest.xml。
- (2) 根据 beanFactoryTest.xml 中的配置找到对应类的配置, 并实例化。
- (3) 调用实例化后的实例。

为了更清楚地描述, 笔者临时画了设计类图, 如图 2-1 所示, 如果想完成我们预想的功能, 至少需要 3 个类。

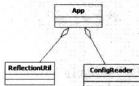


图 2-1 最简单的 Spring 功能架构

- ConfigReader: 用于读取及验证配置文件。我们要用配置文件里面的东西, 当然首先

要做的就是读取，然后放置在内存中。

- ReflectionUtil: 用于根据配置文件中的配置进行反射实例化。比如在上例中 beanFactory/Test.xml 出现的 `<bean id="myTestBean" class="bean.MyTestBean"/>`，我们就可以根据 `bean.MyTestBean` 进行实例化。
- App: 用于完成整个逻辑的串联。

按照最原始的思维方式，整个过程无非如此，但是作为一个风靡世界的优秀源码真的就这么简单吗？

2.3 工程搭建

不如我们首先大致看看 Spring 的源码。在 Spring 源码中，用于实现上面功能的是 `org.springframework.beans.jar`，我们看源码的时候要打开这个工程，如果我们只使用上面的功能，那就没有必要引入 Spring 的其他更多的包，当然 Core 是必须的，还有些依赖的包如图 2-2 所示。

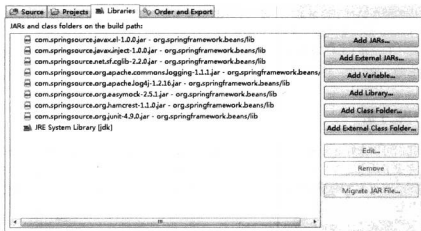


图 2-2 Spring 测试类依赖的 JAR

引入依赖的 JAR 消除掉所有编译错误后，终于可以看源码了。或许你已经知道了答案，Spring 居然用了 N 多代码实现了这个看似很简单的功能，那么这些代码都是做什么用的呢？Spring 在架构或者编码的时候又是如何考虑的呢？带着疑问，让我们踏上了研读 Spring 源码的征程。

2.4 Spring 的结构组成

我们首先尝试梳理一下 Spring 的框架结构,从全局的角度了解一下 Spring 的结构组成。

2.4.1 beans 包的层级结构

笔者认为阅读源码的最好方法是通过示例跟着操作一遍,虽然有时候或者说大多数时候会被复杂的代码绕来绕去,绕到最后已经不知道自己身在何处了,但是,如果配以 UML 还是可以搞定的。笔者就是按照自己的思路进行分析,并配合必要的 UML,希望读者同样可以跟得上思路。

我们先看看整个 beans 工程的源码结构,如图 2-3 所示。

beans 包中的各个源码包的功能如下。

- src/main/java 用于展现 Spring 的主要逻辑。
- src/main/resources 用于存放系统的配置文件。
- src/test/java 用于对主要逻辑进行单元测试。
- src/test/resources 用于存放测试用的配置文件。



图 2-3 beans 工程的源码结构

2.4.2 核心类介绍

通过 beans 工程的结构介绍,我们现在对 beans 的工程结构有了初步的认识,但是在正式开始源码分析之前,有必要了解一下 Spring 中最核心的两个类。

1. DefaultListableBeanFactory

XmlBeanFactory 继承自 DefaultListableBeanFactory,而 DefaultListableBeanFactory 是整个 bean 加载的核心部分,是 Spring 注册及加载 bean 的默认实现,而对于 XmlBeanFactory 与 DefaultListableBeanFactory 不同的地方其实是在 XmlBeanFactory 中使用了自定义的 XML 读取器 XmlBeanDefinitionReader,实现了个性化的 BeanDefinitionReader 读取,DefaultListableBeanFactory 继承了 AbstractAutowireCapableBeanFactory 并实现了 ConfigurableListableBeanFactory 以及 BeanDefinitionRegistry 接口。以下是 ConfigurableListableBeanFactory 的层次结构图(见图 2-4)以及相关类图(见图 2-5)。

从上面的类图以及层次结构图中,我们可以很清晰地从全局角度了解 DefaultListableBeanFactory 的脉络。如果读者没有了解过 Spring 源码可能对上面的类图不是很理解,不过没关系,

通过后续的学习，你会逐渐了解每个类的作用。那么，让我们先简单地了解一下上面类图中的各个类的作用。

- AliasRegistry: 定义对 alias 的简单增删改等操作。
- SimpleAliasRegistry: 主要使用 map 作为 alias 的缓存，并对接口 AliasRegistry 进行实现。
- SingletonBeanRegistry: 定义对单例的注册及获取。
- BeanFactory: 定义获取 bean 及 bean 的各种属性。
- DefaultSingletonBeanRegistry: 对接口 SingletonBeanRegistry 各函数的实现。
- HierarchicalBeanFactory: 继承 BeanFactory，也就是在 BeanFactory 定义的功能的基础上增加了对 parentFactory 的支持。
- BeanDefinitionRegistry: 定义对 BeanDefinition 的各种增删改操作。
- FactoryBeanRegistrySupport: 在 DefaultSingletonBeanRegistry 基础上增加了对 FactoryBean 的特殊处理功能。

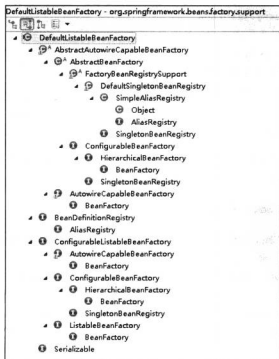


图 2-4 ConfigurableListableBeanFactory 的层次结构图

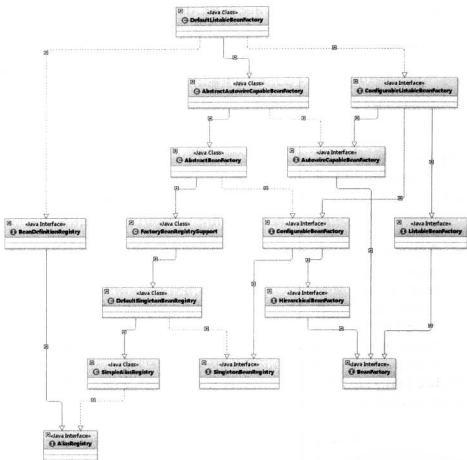


图 2-5 容器加载相关类图

- **ConfigurableBeanFactory**: 提供配置 Factory 的各种方法。
- **ListableBeanFactory**: 根据各种条件获取 bean 的配置清单。
- **AbstractBeanFactory**: 综合 **FactoryBeanRegistrySupport** 和 **ConfigurableBeanFactory** 的功能。
- **AutowireCapableBeanFactory**: 提供创建 bean、自动注入、初始化以及应用 bean 的后处理器。
- **AbstractAutowireCapableBeanFactory**: 综合 **AbstractBeanFactory** 并对接口 **Autowire Capable BeanFactory** 进行实现。

- ConfigurableListableBeanFactory: BeanFactory 配置清单, 指定忽略类型及接口等。
- DefaultListableBeanFactory: 综合上面所有功能, 主要是对 Bean 注册后的处理。

XmlBeanFactory 对 DefaultListableBeanFactory 类进行了扩展, 主要用于从 XML 文档中读取 BeanDefinition, 对于注册及获取 Bean 都是使用从父类 DefaultListableBeanFactory 继承的方法去实现, 而唯独与父类不同的个性化实现就是增加了 XmlBeanDefinitionReader 类型的 reader 属性。在 XmlBeanFactory 中主要使用 reader 属性对资源文件进行读取和注册。

2. XmlBeanDefinitionReader

XML 配置文件的读取是 Spring 中重要的功能, 因为 Spring 的大部分功能都是以配置作为切入点的, 那么我们可以从 XmlBeanDefinitionReader 中梳理一下资源文件读取、解析及注册的大致脉络, 首先我们看看各个类的功能。

- ResourceLoader: 定义资源加载器, 主要应用于根据给定的资源文件地址返回对应的 Resource。
- BeanDefinitionReader: 主要定义资源文件读取并转换为 BeanDefinition 的各个功能。
- EnvironmentCapable: 定义获取 Environment 方法。
- DocumentLoader: 定义从资源文件加载到转换为 Document 的功能。
- AbstractBeanDefinitionReader: 对 EnvironmentCapable、BeanDefinitionReader 类定义的功能进行实现。
- BeanDefinitionDocumentReader: 定义读取 Document 并注册 BeanDefinition 功能。
- BeanDefinitionParserDelegate: 定义解析 Element 的各种方法。

经过以上分析, 我们可以梳理出整个 XML 配置文件读取的大致流程, 如图 2-6 所示, 在 XmlBeanDefinitionReader 中主要包含以下几步的处理。

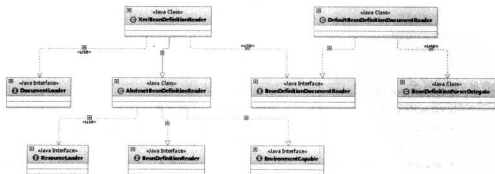


图 2-6 配置文件读取相关类图

- (1) 通过继承自 AbstractBeanDefinitionReader 中的方法, 来使用 ResourceLoader 将资源文件

路径转换为对应的 Resource 文件。

(2) 通过 DocumentLoader 对 Resource 文件进行转换, 将 Resource 文件转换为 Document 文件。

(3) 通过实现接口 BeanDefinitionDocumentReader 的 DefaultBeanDefinitionDocumentReader 类对 Document 进行解析, 并使用 BeanDefinitionParserDelegate 对 Element 进行解析。

2.5 容器的基础 XmlBeanFactory

好了, 到这里我们已经对 Spring 的容器功能有了一个大概的了解, 尽管你可能还很迷糊, 但是不要紧, 接下来我们会详细探索每个步骤的实现。再次重申一下代码, 我们接下来要深入分析以下功能的代码实现:

```
BeanFactory bf = new XmlBeanFactory(new ClassPathResource("beanFactoryTest.xml"));
```

通过 XmlBeanFactory 初始化时序图 (如图 2-7 所示) 我们来看一看上面代码的执行逻辑。

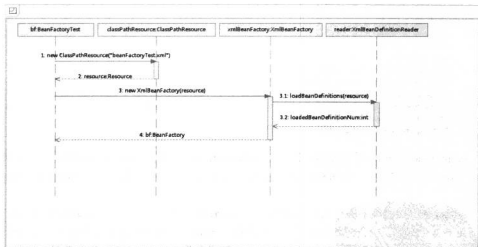


图 2-7 XmlBeanFactory 初始化时序图

时序图从 BeanFactoryTest 测试类开始, 通过时序图我们可以一目了然地看到整个逻辑处理顺序。在测试的 BeanFactoryTest 中首先调用 ClassPathResource 的构造函数来构造 Resource 资源文件的实例对象, 这样后续的资源处理就可以用 Resource 提供的各种服务来操作了, 当我们有了 Resource 后就可以进行 XmlBeanFactory 的初始化了。那么 Resource 资源是如何封装的呢?

2.5.1 配置文件封装

Spring 的配置文件读取是通过 `ClassPathResource` 进行封装的，如 `new ClassPathResource("beanFactoryTest.xml")`，那么 `ClassPathResource` 完成了什么功能呢？

在 Java 中，将不同来源的资源抽象成 `URL`，通过注册不同的 `handler` (`URLStreamHandler`) 来处理不同来源的资源的读取逻辑，一般 `handler` 的类型使用不同前缀（协议，Protocol）来识别，如“file:”、“http:”、“jar:”等，然而 `URL` 没有默认定义相对 `Classpath` 或 `ServletContext` 等资源的 `handler`，虽然可以注册自己的 `URLStreamHandler` 来解析特定的 `URL` 前缀（协议），比如“classpath:”，然而这需要了解 `URL` 的实现机制，而且 `URL` 也没有提供一些基本的方法，如检查当前资源是否存在、检查当前资源是否可读等方法。因而 Spring 对其内部使用到的资源实现了自己的抽象结构：`Resource` 接口来封装底层资源。

```
public interface InputStreamSource {
    InputStream getInputStream() throws IOException;
}

public interface Resource extends InputStreamSource {
    boolean exists();
    boolean isReadable();
    boolean isOpen();
    URL getURL() throws IOException;
    URI getURI() throws IOException;
    File getFile() throws IOException;
    long lastModified() throws IOException;
    Resource createRelative(String relativePath) throws IOException;
    String getFilename();
    String getDescription();
}
```

`InputStreamSource` 封装任何能返回 `InputStream` 的类，比如 `File`、`Classpath` 下的资源和 `Byte Array` 等。它只有一个方法定义：`getInputStream()`，该方法返回一个新的 `InputStream` 对象。

`Resource` 接口抽象了所有 Spring 内部使用到的底层资源：`File`、`URL`、`Classpath` 等。首先，它定义了 3 个判断当前资源状态的方法：存在性（`exists`）、可读性（`isReadable`）、是否处于打开状态（`isOpen`）。另外，`Resource` 接口还提供了不同资源到 `URL`、`URI`、`File` 类型的转换，以及获取 `lastModified` 属性、文件名（不带路径信息的花名，`getFilename()`）的方法。为了便于操作，`Resource` 还提供了基于当前资源创建一个相对资源的方法：`createRelative()`。在错误处理中需要详细地打印出错的资源文件，因而 `Resource` 还提供了 `getDescription()` 方法用于在错误处理中的打印信息。

对不同来源的资源文件都有相应的 `Resource` 实现：文件（`FileSystemResource`）、`Classpath` 资源（`ClassPathResource`）、`URL` 资源（`UrlResource`）、`InputStream` 资源（`InputStreamResource`）、`Byte 数组`（`ByteArrayResource`）等。相关类图如 2-8 所示。

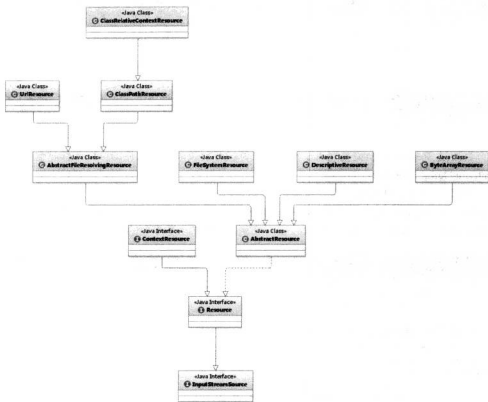


图 2-8 资源文件处理相关类图

在日常的开发工作中，资源文件的加载也是经常用到的，可以直接使用 Spring 提供的类，比如在希望加载文件时可以使用以下代码：

```
Resource resource=new ClassPathResource("beanFactoryTest.xml");
InputStream inputStream=resource.getInputStream();
```

得到 inputStream 后，我们就可以按照以前的开发方式进行实现了，并且我们已经可以利用 Resource 及其子类为我们提供好的诸多特性。

有了 Resource 接口便可以对所有资源文件进行统一处理。至于实现，其实是非常简单的，以 getInputStream 为例，ClassPathResource 中的实现方式便是通过 class 或者 classLoader 提供的底层方法进行调用，而对于 FileSystemResource 的实现其实更简单，直接使用 FileInputStream 对文件进行实例化。

ClassPathResource.java

```

if (this.clazz != null) {
    is = this.clazz.getResourceAsStream(this.path);
} else {
    is = this.classLoader.getResourceAsStream(this.path);
}

```

FileSystemResource.java

```

public InputStream getInputStream() throws IOException {
    return new FileInputStream(this.file);
}

```

当通过 `Resource` 相关类完成了对配置文件进行封装后配置文件的读取工作就全权交给 `XmlBeanDefinitionReader` 来处理了。

了解了 Spring 中将配置文件封装为 `Resource` 类型的实例方法后, 我们就可以继续探寻 `XmlBeanFactory` 的初始化过程了, `XmlBeanFactory` 的初始化有若干办法, Spring 中提供了很多的构造函数, 在这里分析的是使用 `Resource` 实例作为构造函数参数的办法, 代码如下:

XmlBeanFactory.java

```

public XmlBeanFactory(Resource resource) throws BeansException {
    //调用 XmlBeanFactory (Resource, BeanFactory) 构造方法,
    this(resource, null);
}

```

构造函数内部再次调用内部构造函数:

```

//parentBeanFactory 为父类 BeanFactory 用于 factory 合并, 可以为空
public XmlBeanFactory(Resource resource, BeanFactory parentBeanFactory) throws
BeansException {
    super(parentBeanFactory);
    this.reader.loadBeanDefinitions(resource);
}

```

上面函数中的代码 `this.reader.loadBeanDefinitions(resource)` 才是资源加载的真正实现, 也是我们分析的重点之一。我们可以看到时序图中提到的 `XmlBeanDefinitionReader` 加载数据就是在这里完成的, 但是在 `XmlBeanDefinitionReader` 加载数据前还有一个调用父类构造函数初始化的过程: `super(parentBeanFactory)`, 跟踪代码到父类 `AbstractAutowireCapableBeanFactory` 的构造函数中:

AbstractAutowireCapableBeanFactory.java

```

public AbstractAutowireCapableBeanFactory() {
    super();
    ignoreDependencyInterface(BeanNameAware.class);
    ignoreDependencyInterface(BeanFactoryAware.class);
    ignoreDependencyInterface(BeanClassLoaderAware.class);
}

```

这里有必要提及一下 `ignoreDependencyInterface` 方法。 `ignoreDependencyInterface` 的主要功

能是忽略给定接口的自动装配功能,那么,这样做的目的是什么呢?会产生什么样的效果呢?

举例来说,当 A 中有属性 B,那么当 Spring 在获取 A 的 Bean 的时候如果其属性 B 还没有初始化,那么 Spring 会自动初始化 B,这也是 Spring 中提供的一个重要特性。但是,某些情况下, B 不会被初始化,其中的一种情况就是 B 实现了 BeanNameAware 接口。Spring 中是这样介绍的:自动装配时忽略给定的依赖接口,典型应用是通过其他方式解析 Application 上下文注册依赖,类似于 BeanFactory 通过 BeanFactoryAware 进行注入或者 ApplicationContext 通过 ApplicationContextAware 进行注入。

2.5.2 加载 Bean

之前提到的在 XmlBeanFactory 构造函数中调用了 XmlBeanDefinitionReader 类型的 reader 属性提供的方法 this.reader.loadBeanDefinitions(resource),而这句代码则是整个资源加载的切入点,我们先来看看这个方法的时序图,如图 2-9 所示。

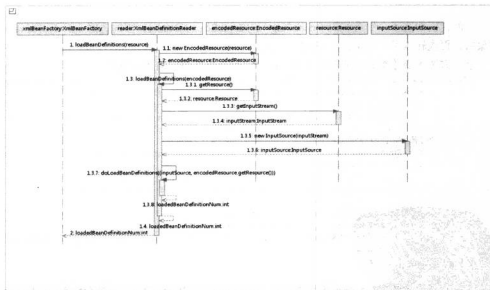


图 2-9 loadBeanDefinitions 函数执行时序图

看到图 2-9 我们才知道什么叫山路十八弯,绕了这么半天还没有真正地切入正题,比如加载 XML 文档和解析注册 Bean,一直还在做准备工作。我们根据上面的时序图来分析一下这里究竟在准备什么?从上面的时序图中我们尝试梳理整个的处理过程如下。

(1) 封装资源文件。当进入 XmlBeanDefinitionReader 后首先对参数 Resource 使用

EncodedResource 类进行封装。

(2) 获取输入流。从 Resource 中获取对应的 InputStream 并构造 InputSource。

(3) 通过构造的 InputSource 实例和 Resource 实例继续调用函数 doLoadBeanDefinitions。

我们来看一下 loadBeanDefinitions 函数具体的实现过程：

```
public int loadBeanDefinitions(Resource resource) throws BeanDefinitionStoreException {
    return loadBeanDefinitions(new EncodedResource(resource));
}
```

那么 EncodedResource 的作用是什么呢？通过名称，我们可以大致推断这个类主要是用于对资源文件的编码进行处理的。其中的主要逻辑体现在 getReader() 方法中，当设置了编码属性的时候 Spring 会使用相应的编码作为输入流的编码。

```
public Reader getReader() throws IOException {
    if (this.encoding != null) {
        return new InputStreamReader(this.resource.getInputStream(), this.encoding);
    }
    else {
        return new InputStreamReader(this.resource.getInputStream());
    }
}
```

上面代码构造了一个有编码 (encoding) 的 InputStreamReader。当构造好 encodedResource 对象后，再次转入了可复用方法 loadBeanDefinitions(new EncodedResource(resource))。

这个方法内部才是真正的数据准备阶段，也就是时序图所描述的逻辑：

```
public int loadBeanDefinitions(EncodedResource encodedResource) throws BeanDefinitionStoreException {
    Assert.notNull(encodedResource, "EncodedResource must not be null");
    if (logger.isInfoEnabled()) {
        logger.info("Loading XML bean definitions from " + encodedResource.getResource());
    }
    //通过属性来记录已经加载的资源
    Set<EncodedResource> currentResources = this.resourcesCurrentlyBeingLoaded.get();
    if (currentResources == null) {
        currentResources = new HashSet<EncodedResource>(4);
        this.resourcesCurrentlyBeingLoaded.set(currentResources);
    }
    if (!currentResources.add(encodedResource)) {
        throw new BeanDefinitionStoreException(
            "Detected cyclic loading of " + encodedResource + " - check your "
            + "import definitions!");
    }
    try {
        //从 encodedResource 中获取已经封装的 Resource 对象并再次从 Resource 中获取其中的 inputStream
        InputStream inputStream = encodedResource.getResource().getInputStream();
        try {
            //InputSource 这个类并不来自于 Spring，它的全路径是 org.xml.sax.InputSource
            InputSource inputSource = new InputSource(inputStream);
            if (encodedResource.getEncoding() != null) {
                inputSource.setEncoding(encodedResource.getEncoding());
            }
            //真正进入了逻辑核心部分
            return doLoadBeanDefinitions(inputSource, encodedResource.getResource());
        }
    }
```

```

    }
    finally {
        //关闭输入流
        inputStream.close();
    }
}
catch (IOException ex) {
    throw new BeanDefinitionStoreException(
        "IOException parsing XML document from " + encodedResource.getResource(), ex);
}
finally {
    currentResources.remove(encodedResource);
    if (currentResources.isEmpty()) {
        this.resourcesCurrentlyBeingLoaded.remove();
    }
}
}
}

```

我们再次整理一下数据准备阶段的逻辑，首先对传入的 `resource` 参数做封装，目的是考虑到 `Resource` 可能存在编码要求的情况，其次，通过 SAX 读取 XML 文件的方式来准备 `InputStream` 对象，最后将准备的数据通过参数传入真正的核心处理部分 `doLoadBeanDefinitions(inputStream, encodedResource.getResource())`。

```

protected int doLoadBeanDefinitions(InputStream inputStream, Resource resource)
    throws BeanDefinitionStoreException {
    try {
        int validationMode = getValidationModeForResource(resource);
        Document doc = this.documentLoader.loadDocument(
            inputStream, getEntityResolver(), this.errorHandler, validationMode,
            isNamespaceAware());
        return registerBeanDefinitions(doc, resource);
    }
    catch (BeanDefinitionStoreException ex) {
        throw ex;
    }
    catch (SAXParseException ex) {
        throw new XmlBeanDefinitionStoreException(resource.getDescription(),
            "Line " + ex.getLineNumber() + " in XML document from " + resource
            + " is invalid", ex);
    }
    catch (SAXException ex) {
        throw new XmlBeanDefinitionStoreException(resource.getDescription(),
            "XML document from " + resource + " is invalid", ex);
    }
    catch (ParserConfigurationException ex) {
        throw new BeanDefinitionStoreException(resource.getDescription(),
            "Parser configuration exception parsing XML from " + resource, ex);
    }
    catch (IOException ex) {
        throw new BeanDefinitionStoreException(resource.getDescription(),
            "IOException parsing XML document from " + resource, ex);
    }
    catch (Throwable ex) {

```

```

        throw new BeanDefinitionStoreException(resource.getDescription(),
            "Unexpected exception parsing XML document from " + resource, ex);
    }
}

```

在上面冗长的代码中假如不考虑异常类的代码，其实只做了三件事，这三件事的每一件都必不可少。

- (1) 获取对 XML 文件的验证模式。
- (2) 加载 XML 文件，并得到对应的 Document。
- (3) 根据返回的 Document 注册 Bean 信息。

这 3 个步骤支撑着整个 Spring 容器部分的实现基础，尤其是第 3 步对配置文件的解析，逻辑非常的复杂，那么我们先从获取 XML 文件的验证模式开始讲起。

2.6 获取 XML 的验证模式

了解 XML 文件的读者都应该知道 XML 文件的验证模式保证了 XML 文件的正确性，而比较常用的验证模式有两种：DTD 和 XSD。它们之间什么区别呢？

2.6.1 DTD 与 XSD 区别

DTD (Document Type Definition) 即文档类型定义，是一种 XML 约束模式语言，是 XML 文件的验证机制，属于 XML 文件组成的一部分。DTD 是一种保证 XML 文档格式正确的有效方法，可以通过比较 XML 文档和 DTD 文件来看文档是否符合规范，元素和标签使用是否正确。一个 DTD 文档包含：元素的定义规则，元素间关系的定义规则，元素可使用的属性，可使用的实体或符号规则。

要使用 DTD 验证模式的时候需要在 XML 文件的头部声明，以下是在 Spring 中使用 DTD 声明方式的代码：

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//Spring//DTD BEAN 2.0//EN" "http://www.springframework.org/dtd/
Spring-beans-2.0.dtd">
<beans>
...
</beans>

```

而以 Spring 为例，具体的 Spring-beans-2.0.dtd 部分如下：

```

<!ELEMENT beans (
    description?,
    (import | alias | bean)*
)>
<!--ATTLIST beans default-lazy-init (true | false) "false">
<!--ATTLIST beans default-merge (true | false) "false">
<!--ATTLIST beans default-autowire (no | byName | byType | constructor | autodetect) "no">
<!--ATTLIST beans default-dependency-check (none | objects | simple | all) "none">
<!--ATTLIST beans default-init-method CDATA #IMPLIED>

```

```
<!ATTLIST beans default-destroy-method CDATA #IMPLIED>
... ..
```

XML Schema 语言就是 XSD (XML Schemas Definition)。XML Schema 描述了 XML 文档的结构。可以用一个指定的 XML Schema 来验证某个 XML 文档,以检查该 XML 文档是否符合其要求。文档设计者可以通过 XML Schema 指定一个 XML 文档所允许的结构和内容,并据此检查一个 XML 文档是否是有效的。XML Schema 本身是一个 XML 文档,它符合 XML 语法规则。可以用通用的 XML 解析器解析它。

在使用 XML Schema 文档对 XML 实例文档进行检验,除了要声明名称空间外 (xmlns=http://www.springframework.org/schema/beans),还必须指定该名称空间所对应的 XML Schema 文档的存储位置。通过 schemaLocation 属性来指定名称空间所对应的 XML Schema 文档的存储位置,它包含两个部分,一部分是名称空间的 URI,另一部分就是该名称空间所标识的 XML Schema 文件位置或 URL 地址 (xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/Spring-beans.xsd")。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/Spring-beans.xsd">
    ... ..
</beans>
```

Spring-beans-3.0.xsd 部分代码如下:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>

<xsd:schema xmlns="http://www.springframework.org/schema/beans"
            xmlns:xsd="http://www.w3.org/2001/XMLSchema"
            targetNamespace="http://www.springframework.org/schema/beans">

    <xsd:import namespace="http://www.w3.org/XML/1998/namespace"/>

    <xsd:annotation>
        <xsd:documentation><![CDATA[
            ... ..
        ]]></xsd:documentation>
    </xsd:annotation>

    <!-- base types -->
    <xsd:complexType name="identifiedType" abstract="true">
        <xsd:annotation>
            <xsd:documentation><![CDATA[
                The unique identifier for a bean. The scope of the identifier
                is the enclosing bean factory.
            ]]></xsd:documentation>
        </xsd:annotation>
        <xsd:attribute name="id" type="xsd:ID">
            <xsd:annotation>
                <xsd:documentation><![CDATA[
                    The unique identifier for a bean.
```

```

]]></xsd:documentation>
</xsd:annotation>
</xsd:attribute>
</xsd:complexType>
... ..
</xsd:schema>

```

我们只是简单地介绍一下 XML 文件的验证模式的相关知识，目的在于让读者对后续知识的理解能有连续性，如果对 XML 有兴趣的读者可以进一步查阅相关资料。

2.6.2 验证模式的读取

了解了 DTD 与 XSD 的区别后我们再去分析 Spring 中对于验证模式的提取就更容易理解了。通过之前的分析我们锁定了 Spring 通过 `getValidationModeForResource` 方法来获取对应资源的验证模式。

```

protected int getValidationModeForResource(Resource resource) {
    int validationModeToUse = getValidationMode();
    //如果手动指定了验证模式则使用指定的验证模式
    if (validationModeToUse != VALIDATION_AUTO) {
        return validationModeToUse;
    }
    //如果未指定则使用自动检测
    int detectedMode = detectValidationMode(resource);
    if (detectedMode != VALIDATION_AUTO) {
        return detectedMode;
    }
    return VALIDATION_XSD;
}

```

方法的实现其实还是很简单的，无非是如果设定了验证模式则使用设定的验证模式（可以通过对调用 `XmlBeanDefinitionReader` 中的 `setValidationMode` 方法进行设定），否则使用自动检测的方式。而自动检测验证模式的功能是在函数 `detectValidationMode` 方法中实现的，在 `detectValidationMode` 函数中又将自动检测验证模式的工作委托给了专门处理类 `XmlValidationModeDetector`，调用了 `XmlValidationModeDetector` 的 `validationModeDetector` 方法，具体代码如下：

```

protected int detectValidationMode(Resource resource) {
    if (resource.isOpen()) {
        throw new BeanDefinitionStoreException(
            "Passed-in Resource [" + resource + "] contains an open stream: " +
            "cannot determine validation mode automatically. Either pass in
a Resource " +
            "that is able to create fresh streams, or explicitly specify the
validationMode " +
            "on your XmlBeanDefinitionReader instance.");
    }

    InputStream inputStream;
    try {
        inputStream = resource.getInputStream();
    }
}

```

```

    }
    catch (IOException ex) {
        throw new BeanDefinitionStoreException(
            "Unable to determine validation mode for [" + resource + "]: cannot
open InputStream. " +
            "Did you attempt to load directly from a SAX InputSource without
specifying the " +
            "validationMode on your XmlBeanDefinitionReader instance?", ex);
    }

    try {
        return this.validationModeDetector.detectValidationMode(inputStream);
    }
    catch (IOException ex) {
        throw new BeanDefinitionStoreException("Unable to determine validation mode for [" +
            resource + "]: an error occurred whilst reading from the
InputStream.", ex);
    }
}

```

XmlValidationModeDetector.java

```

public int detectValidationMode(InputStream inputStream) throws IOException {
    BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
    try {
        boolean isDtdValidated = false;
        String content;
        while ((content = reader.readLine()) != null) {
            content = consumeCommentTokens(content);
            //如果读取的行是空或者是注释则略过
            if (this.inComment || !StringUtils.hasText(content)) {
                continue;
            }
            if (hasDoctype(content)) {
                isDtdValidated = true;
                break;
            }
            //读取到<开始符号，验证模式一定会在开始符号之前
            if (hasOpeningTag(content)) {
                break;
            }
        }
        return (isDtdValidated ? VALIDATION_DTD : VALIDATION_XSD);
    }
    catch (CharConversionException ex) {
        // Choked on some character encoding...
        // Leave the decision up to the caller.
        return VALIDATION_AUTO;
    }
    finally {
        reader.close();
    }
}

```

```
private boolean hasDoctype(String content) {  
    return (content.indexOf(DOCTYPE) > -1);  
}
```

只要我们理解了 XSD 与 DTD 的使用方法,理解上面的代码应该不会太难, Spring 用来检测验证模式的办法就是判断是否包含 DOCTYPE, 如果包含就是 DTD, 否则就是 XSD。

2.7 获取 Document

经过了验证模式准备的步骤就可以进行 Document 加载了, 同样 XmlBeanFactoryReader 类对于文档读取并没有亲力亲为, 而是委托给了 DocumentLoader 去执行, 这里的 DocumentLoader 是个接口, 而真正调用的是 DefaultDocumentLoader, 解析代码如下:

```
DefaultDocumentLoader.java  
public Document loadDocument(InputSource inputSource, EntityResolver entityResolver,  
    ErrorHandler errorHandler, int validationMode, boolean namespaceAware) throws  
Exception {  
    DocumentBuilderFactory factory = createDocumentBuilderFactory(validationMode,  
namespaceAware);  
    if (logger.isDebugEnabled()) {  
        logger.debug("Using JAXP provider [" + factory.getClass().getName() + "]);  
    }  
    DocumentBuilder builder = createDocumentBuilder(factory, entityResolver,  
errorHandler);  
    return builder.parse(inputSource);  
}
```

对于这部分代码其实并没有太多可以描述的, 因为通过 SAX 解析 XML 文档的套路大致都差不多, Spring 在这里并没有什么特殊的地方, 同样首先创建 DocumentBuilderFactory, 再通过 DocumentBuilderFactory 创建 DocumentBuilder, 进而解析 inputSource 来返回 Document 对象。对此感兴趣的读者可以在网上获取更多的资料。这里有必要提及一下 EntityResolver, 对于参数 entityResolver, 传入的是通过 getEntityResolver() 函数获取的返回值, 如下代码:

```
protected EntityResolver getEntityResolver() {  
    if (this.entityResolver == null) {  
        // Determine default EntityResolver to use.  
        ResourceLoader resourceLoader = getResourceLoader();  
        if (resourceLoader != null) {  
            this.entityResolver = new ResourceEntityResolver(resourceLoader);  
        }  
        else {  
            this.entityResolver = new DelegatingEntityResolver(getBeanClassLoader());  
        }  
    }  
    return this.entityResolver;  
}
```

那么, EntityResolver 到底是做什么用的呢?

2.7.1 EntityResolver 用法

在 `loadDocument` 方法中涉及一个参数 `EntityResolver`，何为 `EntityResolver`？官网这样解释：如果 SAX 应用程序需要实现自定义处理外部实体，则必须实现此接口并使用 `setEntityResolver` 方法向 SAX 驱动器注册一个实例。也就是说，对于解析一个 XML，SAX 首先读取该 XML 文档上的声明，根据声明去寻找相应的 DTD 定义，以便对文档进行一个验证。默认的寻找规则，即通过网络（实现上就是声明的 DTD 的 URI 地址）来下载相应的 DTD 声明，并进行认证。下载的过程是一个漫长的过程，而且当网络中断或不可用时，这里会报错，就是因为相应的 DTD 声明没有被找到的原因。

`EntityResolver` 的作用是项目本身就可以提供一个如何寻找 DTD 声明的方法，即由程序来实现寻找 DTD 声明的过程，比如我们将 DTD 文件放到项目中某处，在实现时直接将此文档读取并返回给 SAX 即可。这样就避免了通过网络来寻找相应的声明。

首先看 `entityResolver` 的接口方法声明：

```
InputSource resolveEntity(String publicId, String systemId)
```

这里，它接收两个参数 `publicId` 和 `systemId`，并返回一个 `inputSource` 对象。这里我们以特定配置文件来进行讲解。

(1) 如果我们在解析验证模式为 XSD 的配置文件，代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/Spring-beans.xsd">
    ...
</beans>
```

读取到以下两个参数。

● **publicId:** null

● **systemId:** `http://www.springframework.org/schema/beans/Spring-beans.xsd`

(2) 如果我们在解析验证模式为 DTD 的配置文件，代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//Spring//DTD BEAN 2.0//EN" "http://www.springframework.org/dtd/Spring-beans-2.0.dtd">
<beans>
    ...
</beans>
```

读取到以下两个参数。

● **publicId:** `-//Spring//DTD BEAN 2.0//EN`

● **systemId:** `http://www.springframework.org/dtd/Spring-beans-2.0.dtd`

之前已经提到过，验证文件默认的加载方式是通过 URL 进行网络下载获取，这样会造成延迟，用户体验也不好，一般的做法都是将验证文件放置在自己的工程里，那么怎么做才能将这个 URL 转换为自己工程里对应的地址文件呢？我们以加载 DTD 文件为例来看看 Spring 中是

如何实现的。根据之前 Spring 中通过 `getEntityResolver()` 方法对 `EntityResolver` 的获取，我们知道，Spring 中使用 `DelegatingEntityResolver` 类为 `EntityResolver` 的实现类，`resolveEntity` 实现方法如下：

```
DelegatingEntityResolver.java

public InputSource resolveEntity(String publicId, String systemId) throws
SAXException, IOException {
    if (systemId != null) {
        if (systemId.endsWith(DTD_SUFFIX)) {
            //如果是 dtd 从这里解析
            return this.dtdResolver.resolveEntity(publicId, systemId);
        }
        else if (systemId.endsWith(XSD_SUFFIX)) {
            //通过调用 META-INF/Spring.schemas 解析
            return this.schemaResolver.resolveEntity(publicId, systemId);
        }
    }
    return null;
}
```

我们可以看到，对不同的验证模式，Spring 使用了不同的解析器解析。这里简单描述一下原理，比如加载 DTD 类型的 `BeansDtdResolver` 的 `resolveEntity` 是直接截取 `systemId` 最后的 `xx.dtd` 然后去当前路径下寻找，而加载 XSD 类型的 `PluggableSchemaResolver` 类的 `resolveEntity` 是默认到 `META-INF/Spring.schemas` 文件中找到 `systemid` 所对应的 XSD 文件并加载。

```
BeansDtdResolver.java

public InputSource resolveEntity(String publicId, String systemId) throws IOException {
    if (logger.isTraceEnabled()) {
        logger.trace("Trying to resolve XML entity with public ID [" + publicId +
            "] and system ID [" + systemId + "];");
    }
    // DTD_EXTENSION = ".dtd";
    if (systemId != null && systemId.endsWith(DTD_EXTENSION)) {
        int lastPathSeparator = systemId.lastIndexOf("/");
        for (String DTD_NAME : DTD_NAMES) {
            // DTD_NAMES = {"Spring-beans-2.0", "Spring-beans"};
            int dtdNameStart = systemId.indexOf(DTD_NAME);
            if (dtdNameStart > lastPathSeparator) {
                String dtdFile = systemId.substring(dtdNameStart);
                if (logger.isTraceEnabled()) {
                    logger.trace("Trying to locate [" + dtdFile + "] in Spring jar");
                }
                try {
                    Resource resource = new ClassPathResource(dtdFile, getClass());
                    InputSource source = new InputSource(resource.getInputStream());
                    source.setPublicId(publicId);
                    source.setSystemId(systemId);
                    if (logger.isDebugEnabled()) {
                        logger.debug("Found beans DTD [" + systemId + "] in
```



```

        this.readerContext = readerContext;

        logger.debug("Loading bean definitions");
        Element root = doc.getDocumentElement();

        doRegisterBeanDefinitions(root);
    }

```

经过艰难险阻，磕磕绊绊，我们终于到了核心逻辑的底部 `doRegisterBeanDefinitions(root)`，至少我们在这个方法中看到了希望。

如果说以前一直是 XML 加载解析的准备阶段，那么 `doRegisterBeanDefinitions` 算是真正地开始进行解析了，我们期待的核心部分真正开始了。

```

protected void doRegisterBeanDefinitions(Element root) {
    //处理 profile 属性
    String profileSpec = root.getAttribute(PROFILE_ATTRIBUTE);
    if (StringUtils.hasText(profileSpec)) {
        Assert.state(this.environment != null, "environment property must not be null");
        String[] specifiedProfiles = StringUtils.tokenizeToStringArray(profileSpec,
            BeanDefinitionParserDelegate.MULTI_VALUE_ATTRIBUTE_DELIMITERS);
        if (!this.environment.acceptsProfiles(specifiedProfiles)) {
            return;
        }
    }
    //专门处理解析
    BeanDefinitionParserDelegate parent = this.delegate;
    this.delegate = createHelper(readerContext, root, parent);

    //解析前处理，留给子类实现
    preProcessXml(root);
    parseBeanDefinitions(root, this.delegate);
    //解析后处理，留给子类实现
    postProcessXml(root);

    this.delegate = parent;
}

```

通过上面的代码我们看到了处理流程，首先是对 `profile` 的处理，然后开始进行解析，可是当我们跟进 `preProcessXml(root)` 或者 `postProcessXml(root)` 发现代码是空的，既然是空的写着还有什么用呢？就像面向对象设计方法学中常说的一句话，一个类要么是面向继承的设计的，要么就用 `final` 修饰。在 `DefaultBeanDefinitionDocumentReader` 中并没有用 `final` 修饰，所以它是面向继承而设计的。这两个方法正是为子类而设计的，如果读者有了了解过设计模式，可以很快速地反映出这是模版方法模式，如果继承自 `DefaultBeanDefinitionDocumentReader` 的子类需要在 `Bean` 解析前后做一些处理的话，那么只需要重写这两个方法就可以了。

2.8.1 profile 属性的使用

我们注意到在注册 `Bean` 的最开始是对 `PROFILE_ATTRIBUTE` 属性的解析，可能对于我们来说，`profile` 属性并不是很常用。让我们先了解一下这个属性。

分析 `profile` 前我们先了解下 `profile` 的用法，官方示例代码片段如下：

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:jdbc="http://www.
Springframework.org/schema/jdbc"
       xmlns:jee="http://www.springframework.org/schema/jee"
       xsi:schemaLocation="...">
    ...
<beans profile="dev">
    ...
</beans>
<beans profile="production">
    ...
</beans>
</beans>

```

集成到 Web 环境中时，在 web.xml 中加入以下代码：

```

<context-param>
<param-name>Spring.profiles.active</param-name>
<param-value>dev</param-value>
</context-param>

```

有了这个特性我们就可以同时在配置文件中部署两套配置来适用于生产环境和开发环境，这样可以方便的进行切换开发、部署环境，最常用的就是更换不同的数据库。

了解了 profile 的使用再来分析代码会清晰得多，首先程序会获取 beans 节点是否定义了 profile 属性，如果定义了则会需要到环境变量中去找，所以这里首先断言 environment 可能为空，因为 profile 是可以同时指定多个的，需要程序对其拆分，并解析每个 profile 是都符合环境变量中所定义的，不定义则不会浪费性能去解析。

2.8.2 解析并注册 BeanDefinition

处理了 profile 后就可以进行 XML 的读取了，跟踪代码进入 parseBeanDefinitions(root, this.delegate)。

```

protected void parseBeanDefinitions(Element root, BeanDefinitionParserDelegate delegate) {
    //对 beans 的处理
    if (delegate.isDefaultNamespace(root)) {
        NodeList nl = root.getChildNodes();
        for (int i = 0; i < nl.getLength(); i++) {
            Node node = nl.item(i);
            if (node instanceof Element) {
                Element ele = (Element) node;
                if (delegate.isDefaultNamespace(ele)) {
                    //对 bean 的处理
                    parseDefaultElement(ele, delegate);
                }
                else {
                    //对 bean 的处理
                    delegate.parseCustomElement(ele);
                }
            }
        }
    }
}

```

```
        else {  
            delegate.parseCustomElement(root);  
        }  
    }  
}
```

上面的代码看起来逻辑还是蛮清晰的，因为在 Spring 的 XML 配置里面有两大类 Bean 声明，一个是默认的，如：

```
<bean id="test" class="test.TestBean"/>
```

另一类就是自定义的，如：

```
<tx:annotation-driven/>
```

而两种方式的读取及解析差别是非常大的，如果采用 Spring 默认的配置，Spring 当然知道该怎么做，但是如果是自定义的，那么就需要用户实现一些接口及配置了。对于根节点或者子节点如果是默认命名空间的话则采用 `parseDefaultElement` 方法进行解析，否则使用 `delegate.parseCustomElement` 方法对自定义命名空间进行解析。而判断是否默认命名空间还是自定义命名空间的办法其实是使用 `node.getNamespaceURI()` 获取命名空间，并与 Spring 中固定的命名空间 <http://www.springframework.org/schema/beans> 进行比对。如果一致则认为是默认，否则就认为是自定义。而对于默认标签解析与自定义标签解析我们将会在下一章中进行讨论。

第3章 默认标签的解析

之前提到过 Spring 中的标签包括默认标签和自定义标签两种，而两种标签的用法以及解析方式存在着很大的不同，本章重点带领读者详细分析默认标签的解析过程。

默认标签的解析是在 `parseDefaultElement` 函数中进行的，函数中的功能逻辑一目了然，分别对 4 种不同标签（`import`、`alias`、`bean` 和 `beans`）做了不同的处理。

```
private void parseDefaultElement(Element ele, BeanDefinitionParserDelegate delegate) {
    //对 import 标签的处理
    if (delegate.nodeNameEquals(ele, IMPORT_ELEMENT)) {
        importBeanDefinitionResource(ele);
    }
    //对 alias 标签的处理
    else if (delegate.nodeNameEquals(ele, ALIAS_ELEMENT)) {
        processAliasRegistration(ele);
    }
    //对 bean 标签的处理
    else if (delegate.nodeNameEquals(ele, BEAN_ELEMENT)) {
        processBeanDefinition(ele, delegate);
    }
    //对 beans 标签的处理
    else if (delegate.nodeNameEquals(ele, NESTED_BEANS_ELEMENT)) {
        doRegisterBeanDefinitions(ele);
    }
}
```

3.1 bean 标签的解析及注册

在 4 种标签的解析中，对 `bean` 标签的解析最为复杂也最为重要，所以我们从此标签开始深入分析，如果能理解此标签的解析过程，其他标签的解析自然会迎刃而解。首先我们进入函数 `processBeanDefinition(ele, delegate)`。

```
protected void processBeanDefinition(Element ele, BeanDefinitionParserDelegate delegate) {
    BeanDefinitionHolder bdHolder = delegate.parseBeanDefinitionElement(ele);
    if (bdHolder != null) {
```

```

        bdHolder = delegate.decorateBeanDefinitionIfRequired(ele, bdHolder);
        try {
            BeanDefinitionReaderUtils.registerBeanDefinition(bdHolder, getReaderContext().
getRegistry());
        }
        catch (BeanDefinitionStoreException ex) {
            getReaderContext().error("Failed to register bean definition with name '" +
                bdHolder.getBeanName() + "'", ele, ex);
        }
        getReaderContext().fireComponentRegistered(new BeanComponentDefinition(bdHolder));
    }
}

```

乍一看，似乎一头雾水，没有以前的函数那样清晰的逻辑。大致的逻辑总结如下。

(1) 首先委托 `BeanDefinitionDelegate` 类的 `parseBeanDefinitionElement` 方法进行元素解析，返回 `BeanDefinitionHolder` 类型的实例 `bdHolder`，经过这个方法后，`bdHolder` 实例已经包含我们配置文件中配置的各种属性了，例如 `class`、`name`、`id`、`alias` 之类的属性。

(2) 当返回的 `bdHolder` 不为空的情况下若存在默认标签的子节点下再有自定义属性，还需要再次对自定义标签进行解析。

(3) 解析完成后，需要对解析后的 `bdHolder` 进行注册，同样，注册操作委托给了 `BeanDefinitionReaderUtils` 的 `registerBeanDefinition` 方法。

(4) 最后发出响应事件，通知想关的监听器，这个 `bean` 已经加载完成了。

配合时序图（如图 3-1 所示），可能会更容易理解。

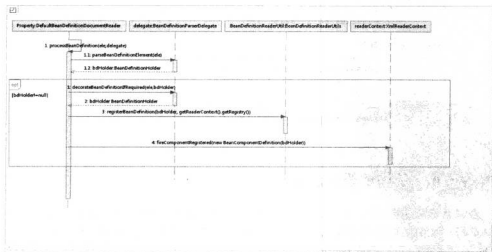


图 3-1 bean 标签的解析及注册时序图

3.1.1 解析 BeanDefinition

下面我们就针对各个操作做具体分析。首先我们从元素解析及信息提取开始，也就是 `BeanDefinitionHolder bdHolder = delegate.parseBeanDefinitionElement(ele)`，进入 `BeanDefinitionDelegate` 类的 `parseBeanDefinitionElement` 方法。

```

BeanDefinitionDelegate.java

public BeanDefinitionHolder parseBeanDefinitionElement(Element ele) {
    return parseBeanDefinitionElement(ele, null);
}

public BeanDefinitionHolder parseBeanDefinitionElement(Element ele, BeanDefinition
containingBean) {
    //解析 id 属性
    String id = ele.getAttribute(ID_ATTRIBUTE);
    //解析 name 属性
    String nameAttr = ele.getAttribute(NAME_ATTRIBUTE);

    //分割 name 属性
    List<String> aliases = new ArrayList<String>();
    if (StringUtils.hasLength(nameAttr)) {
        String[] nameArr = StringUtils.tokenizeToStringArray(nameAttr, MULTI_
VALUE_ATTRIBUTE_DELIMITERS);
        aliases.addAll(Arrays.asList(nameArr));
    }

    String beanName = id;
    if (!StringUtils.hasText(beanName) && !aliases.isEmpty()) {
        beanName = aliases.remove(0);
        if (logger.isDebugEnabled()) {
            logger.debug("No XML 'id' specified - using '" + beanName +
                "' as bean name and " + aliases + " as aliases");
        }
    }

    if (containingBean == null) {
        checkNameUniqueness(beanName, aliases, ele);
    }

    AbstractBeanDefinition beanDefinition = parseBeanDefinitionElement(ele, beanName,
containingBean);
    if (beanDefinition != null) {
        if (!StringUtils.hasText(beanName)) {
            try {
                //如果不存在 beanName 那么根据 Spring 中提供的命名规则为当前 bean 生成对应
                的 beanName

                if (containingBean != null) {
                    beanName = BeanDefinitionReaderUtils.generateBeanName(
                        beanDefinition, this.readerContext.getRegistry(), true);
                }
            }
        }
    }
}

```

```

        else {
            beanName = this.readerContext.generateBeanName(beanDefinition);

            String beanClassName = beanDefinition.getBeanClassName();
            if (beanClassName != null &&
                beanName.startsWith(beanClassName) && beanName.length()
                    > beanClassName.length() &&
                    !this.readerContext.getRegistry().isBeanNameInUse(
                        beanClassName)) {
                aliases.add(beanClassName);
            }
        }
        if (logger.isDebugEnabled()) {
            logger.debug("Neither XML 'id' nor 'name' specified - " +
                "using generated bean name [" + beanName + "];");
        }
    }
    catch (Exception ex) {
        error(ex.getMessage(), ele);
        return null;
    }
}

String[] aliasesArray = StringUtils.toStringArray(aliases);
return new BeanDefinitionHolder(beanDefinition, beanName, aliasesArray);
}

return null;
}

```

以上便是对默认标签解析的全过程了。当然，对 Spring 的解析犹如洋葱剥皮一样，一层一层地进行，尽管现在只能看到对属性 `id` 以及 `name` 的解析，但是很庆幸，思路我们已经了解了。在开始对属性展开全面解析前，Spring 在外层又做了一个当前层的功能架构，在当前层完成的主要工作包括如下内容。

- (1) 提取元素中的 `id` 以及 `name` 属性。
- (2) 进一步解析其他所有属性并统一封装至 `GenericBeanDefinition` 类型的实例中。
- (3) 如果检测到 `bean` 没有指定 `beanName`，那么使用默认规则为此 `Bean` 生成 `beanName`。
- (4) 将获取到的信息封装到 `BeanDefinitionHolder` 的实例中。

我们进一步地查看步骤 (2) 中对标签其他属性的解析过程。

```

public AbstractBeanDefinition parseBeanDefinition(
    Element ele, String beanName, BeanDefinition containingBean) {

    this.parseState.push(new BeanEntry(beanName));

    String className = null;
    //解析 class 属性
    if (ele.hasAttribute(CLASS_ATTRIBUTE)) {
        className = ele.getAttribute(CLASS_ATTRIBUTE).trim();
    }

    try {

```

```

String parent = null;
//解析 parent 属性
if (ele.hasAttribute(PARENT_ATTRIBUTE)) {
    parent = ele.getAttribute(PARENT_ATTRIBUTE);
}
//创建用于承载属性的 AbstractBeanDefinition 类型的 GenericBeanDefinition
AbstractBeanDefinition bd = createBeanDefinition(className, parent);

//硬编码解析默认 bean 的各种属性
parseBeanDefinitionAttributes(ele, beanName, containingBean, bd);
//提取 description
bd.setDescription(DomUtils.getChildElementValueByTagName(ele, DESCRIPTION_
ELEMENT));

//解析元数据
parseMetaElements(ele, bd);
//解析 lookup-method 属性
parseLookupOverrideSubElements(ele, bd.getMethodOverrides());
//解析 replaced-method 属性
parseReplacedMethodSubElements(ele, bd.getMethodOverrides());

//解析构造函数参数
parseConstructorArgElements(ele, bd);
//解析 property 子元素
parsePropertyElements(ele, bd);
//解析 qualifier 子元素
parseQualifierElements(ele, bd);

bd.setResource(this.readerContext.getResource());
bd.setSource(extractSource(ele));

return bd;
}
catch (ClassNotFoundException ex) {
    error("Bean class [" + className + "] not found", ele, ex);
}
catch (NoClassDefFoundError err) {
    error("Class that bean class [" + className + "] depends on not found", ele, err);
}
catch (Throwable ex) {
    error("Unexpected failure during bean definition parsing", ele, ex);
}
finally {
    this.parseState.pop();
}

return null;
}

```

终于，bean 标签的所有属性，不论常用的还是不常用的我们都看到了，尽管有些复杂的属性还需要进一步的解析，不过丝毫不会影响我们兴奋的心情。接下来，我们继续一些复杂标签属性的解析。

1. 创建用于属性承载的 BeanDefinition

BeanDefinition 是一个接口，在 Spring 中存在三种实现：RootBeanDefinition、ChildBeanDefinition 以及 GenericBeanDefinition。三种实现均继承了 AbstractBeanDefinition，其中 BeanDefinition 是配置文件<bean>元素标签在容器中的内部表示形式。<bean>元素标签拥有 class、scope、lazy-init 等配置属性，BeanDefinition 则提供了相应的 beanClass、scope、lazyInit 属性，BeanDefinition 和<bean>中的属性是一一对应的。其中 RootBeanDefinition 是最常用的实现类，它对应一般性的<bean>元素标签，GenericBeanDefinition 是自 2.5 版本以后新加入的 bean 文件配置属性定义类，是一站式服务类。

在配置文件中可以定义父<bean>和子<bean>，父<bean>用 RootBeanDefinition 表示，而子<bean>用 ChildBeanDefinition 表示，而没有父<bean>的<bean>就使用 RootBeanDefinition 表示。AbstractBeanDefinition 对两者共同的类信息进行抽象。

Spring 通过 BeanDefinition 将配置文件中的<bean>配置信息转换为容器的内部表示，并将这些 BeanDefinition 注册到 BeanDefinitionRegistry 中。Spring 容器的 BeanDefinitionRegistry 就像是 Spring 配置信息的内存数据库，主要是以 map 的形式保存，后续操作直接从 BeanDefinition Registry 中读取配置信息。它们之间的关系如图 3-2 所示。

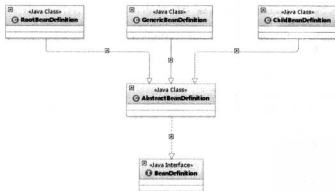


图 3-2 BeanDefinition 及其实现类

由此可知，要解析属性首先要创建用于承载属性的实例，也就是创建 GenericBeanDefinition 类型的实例。而代码 createBeanDefinition(className, parent)的作用就是实现此功能。

protected AbstractBeanDefinition createBeanDefinition(String className, String parentName)
throws ClassNotFoundException {

```

    return BeanDefinitionReaderUtils.createBeanDefinition(
        parentName, className, this.readerContext.getBeanClassLoader());
}

```

有关此电子图书的说明

本人由于一些便利条件,可以帮您提供各种中文电子图书资料,且质量均为清晰的 PDF 图片格式,质量要高于网上大量传播的一些超星 PDG 的图书。方便阅读和携带。只要图书不是太新,文学、法律、计算机、人文、经济、医学、工业、学术等方面的图书,我都可以帮您找到电子版本。所以,当你想要看什么图书时,可以联系我。我的 QQ 是: 85013855,大家可以在 QQ 上联系我。

此 PDF 文件为本人亲自制作,请各位爱书之人尊重个人劳动,敬请您不要修改此 PDF 文件。因为这些图书都是有版权的,请各位怜惜电子图书资源,不要随意传播,否则,这些资源更难以得到。